W2117030
Matey Mateev

# 3DIMD Project Report

**Introduction**

Matey Mateev w2117030 – Group 9, Student 4

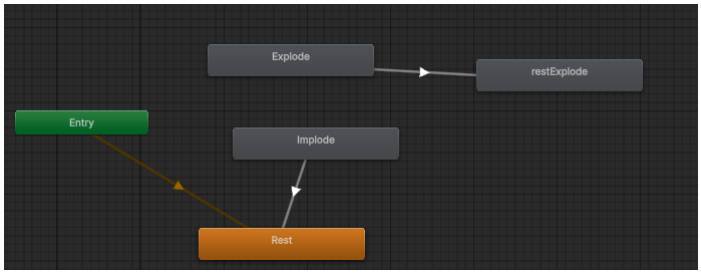
I have been assigned student 4 in this coursework, which means that I will be in charge of building the exploding brick array with key triggers and a minifig with a button-activated walk animation

**Functionality**

Everything in my scene works with buttons and triggers. The UI is composed of 3 (or 2) buttons "Bricks" which takes the user to the exploding brick array, "minifig" which takes the user to the animated mini-figure and "walk/stand" which triggers the animations for the mini-figure. Walk/Stand is toggled by the minifig and bricks buttons, meaning it only shows up when on the minifig screen. This is handled by a script that uses the .SetActive function for unity buttons to toggle its visibility.

Brick array

The brick array is laid out like so; The array itself, and then 2 2x2 lego bricks in front of it with different colours. These are the triggers used for the animations. When the user clicks the red brick, the brick array "explodes" and holds the exploded shape until the user clicks the yellow brick, which then returns the bricks back to their original place.



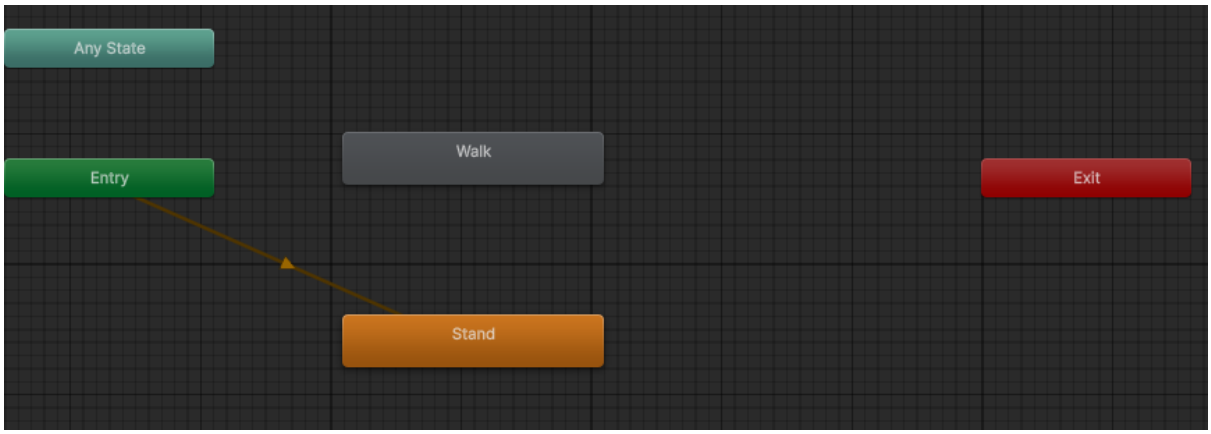

This is the BrickArray animator, The rest animation is set as the default, with the explode animation being linked to the restExplode animation (this allows the array to hold the exploded shape once the red trigger is pressed), whereas the implode animation is linked to the resting animation so that the array stays in its imploded state until the explode animation is called.

The two triggers are handled by 2 separate explode and implode scripts, one using a tag for the brickarray object. These scripts to very similar things, they wait for a click on the trigger's collider and call the respective animations on the brickArray

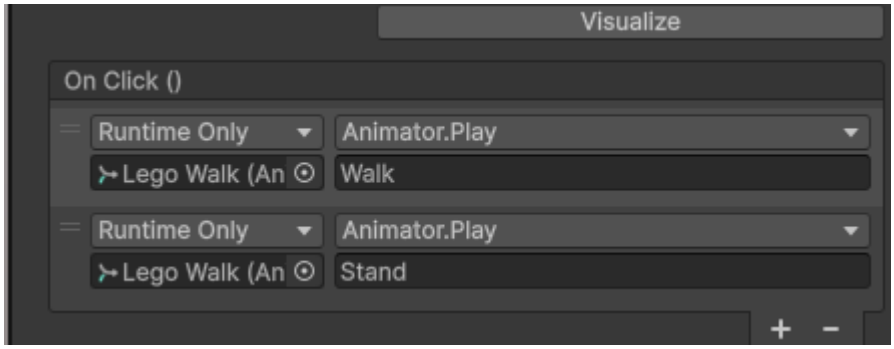
W2117030
Matey Mateev

Animated Minifig

The minifig is much simpler, it has a separate walking and standing animations, with standing being set as the default.



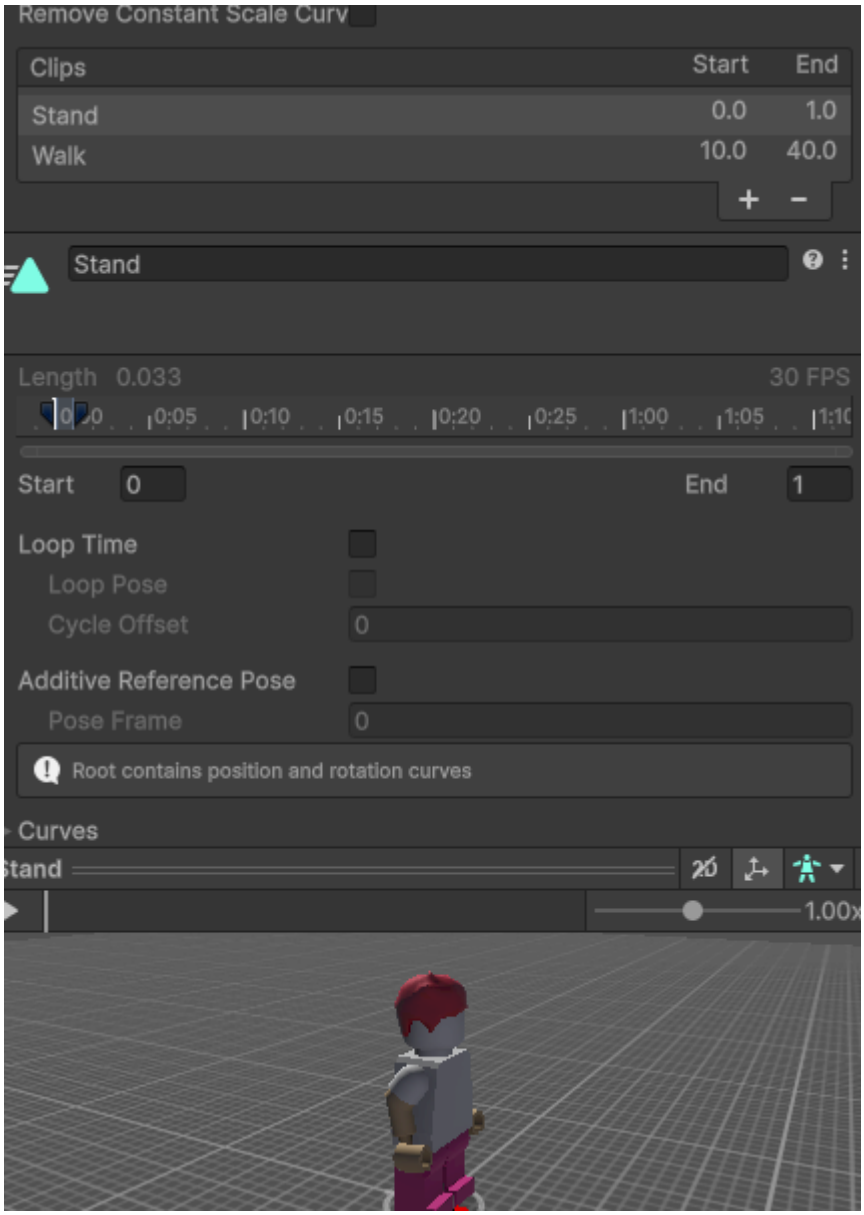

A button located at the bottom of the screen has 2 onclick() functions that use the animator.play function to call the walk and stand animations, meaning that they are a toggle, so if the character is walking, pressing the button would stop the animation and vice versa.



W2117030
Matey Mateev

**(Development Log) Minifig Animation**

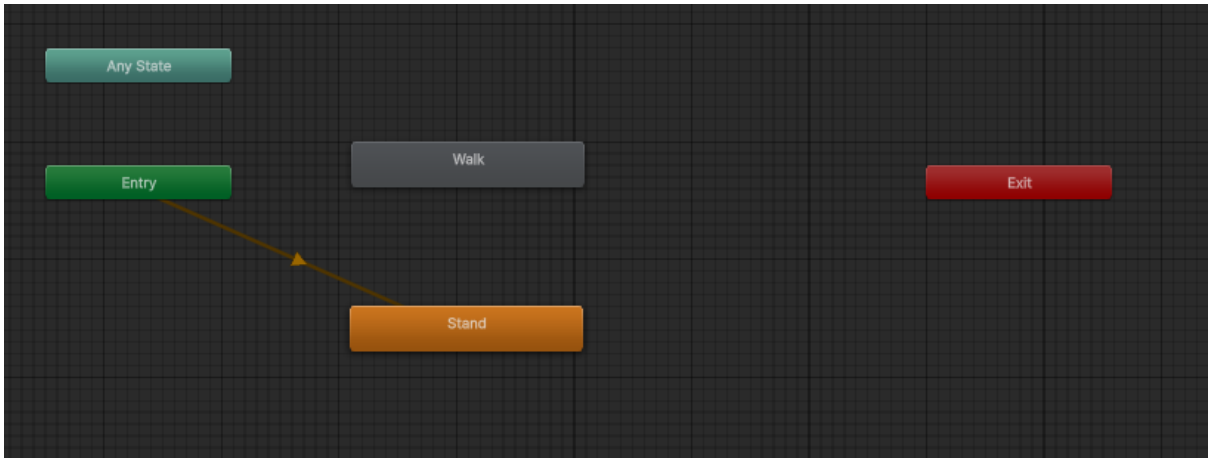
I started by importing the minifig model from blackboard and creating the standing/walk animations.



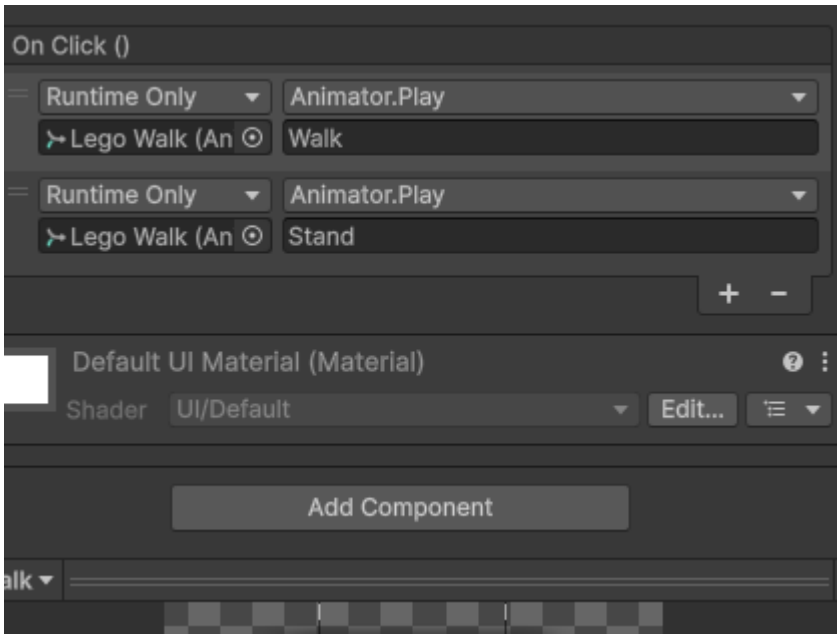

I then created an animation controller for the minifig.

W2117030
Matey Mateev

I added the animations to the minifig's animator, making the stand be the default animation that plays




After this I created a button located on the bottom of the screen, adding an onclick() function to toggle the walk and stand animations when pressed




An error I encountered was when I pressed play, the lego figure would shift to a different place before playing the animation. This is because the object had position keyframes. To fix this I simply shifted the base position of the minifig to 0,0.5,0 to match the keyframes so that the figure stayed in place.

W2117030
Matey Mateev




The lego minifig walking



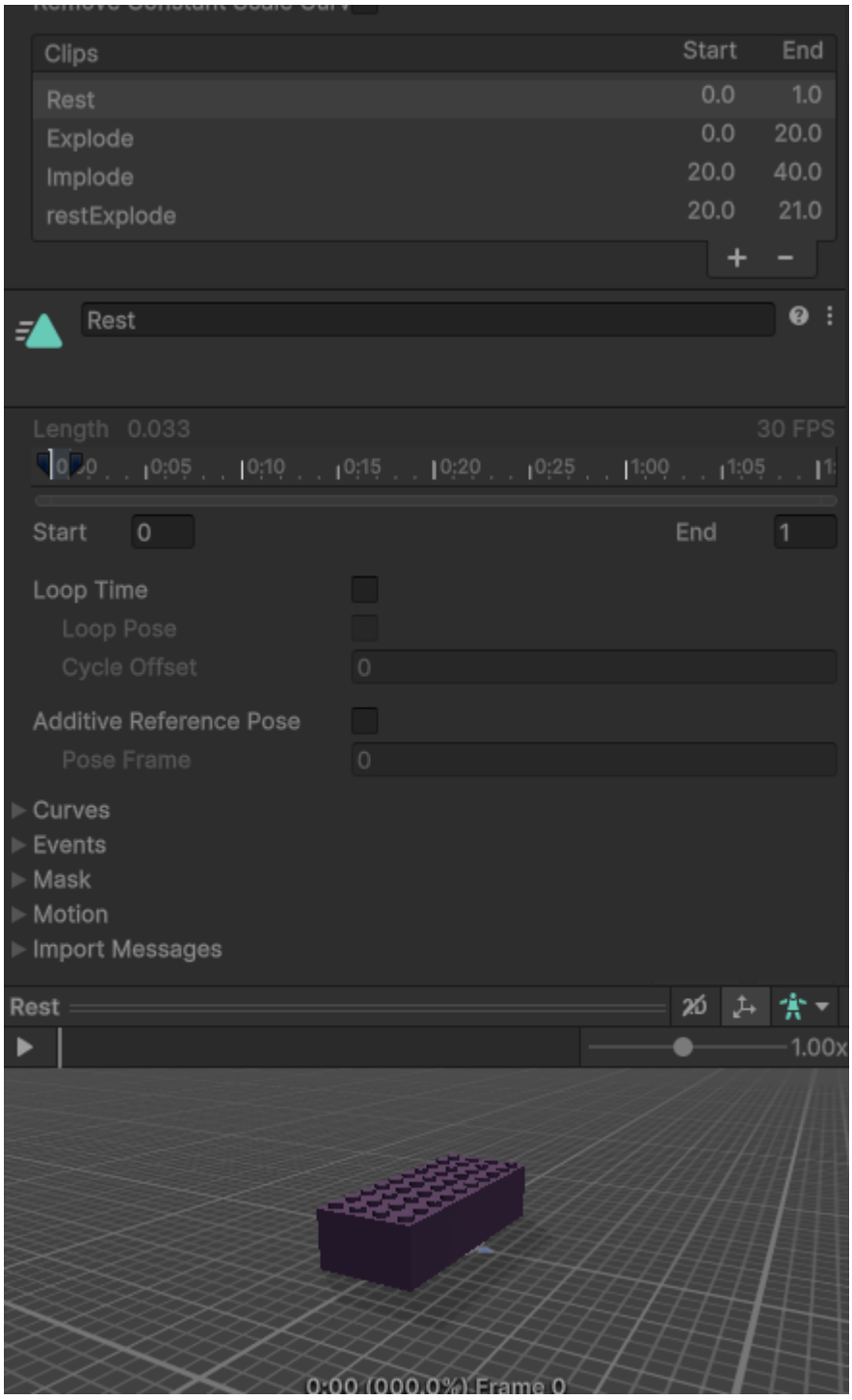

The lego minifig standing

**(Development Log) Brick Array and Triggers**

W2117030
Matey Mateev

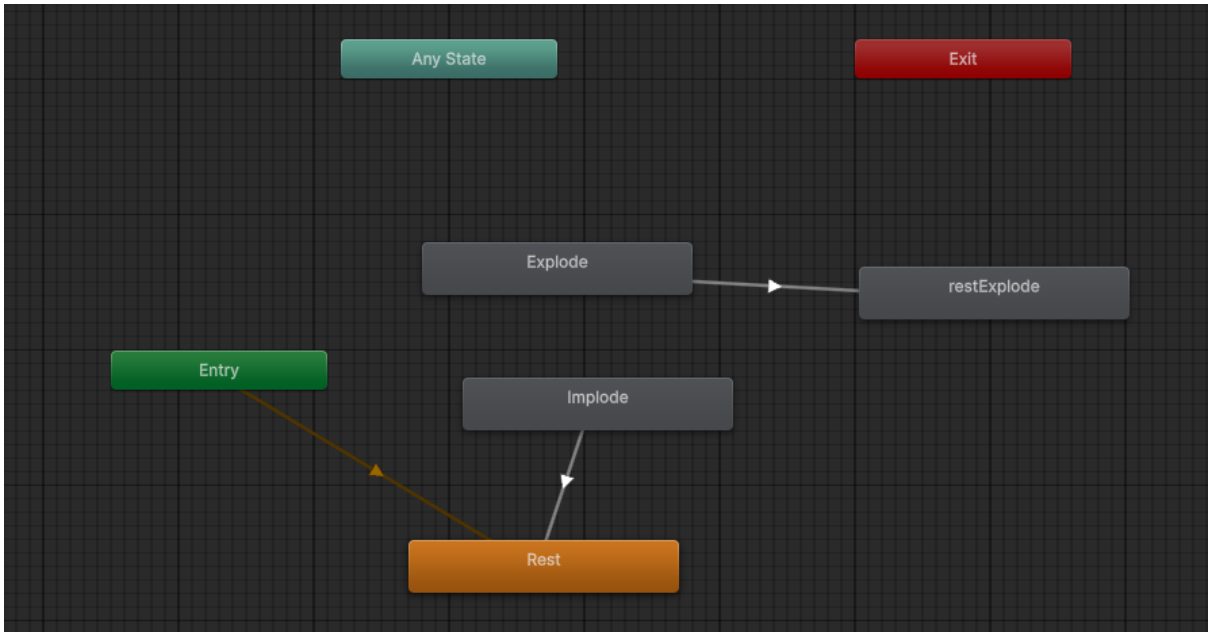
I uploaded the eight brick array from blackboard into my unity project and moved it away to a different position. I created a new camera to point towards this object.

Like the minifig, I first created the animations for the exploding and imploding, I needed more here due to the fact that the object required to keep its exploded/imploded state until the other trigger was pressed. I ended up with 4 animations in Rest,Explode,Implode and restExplode

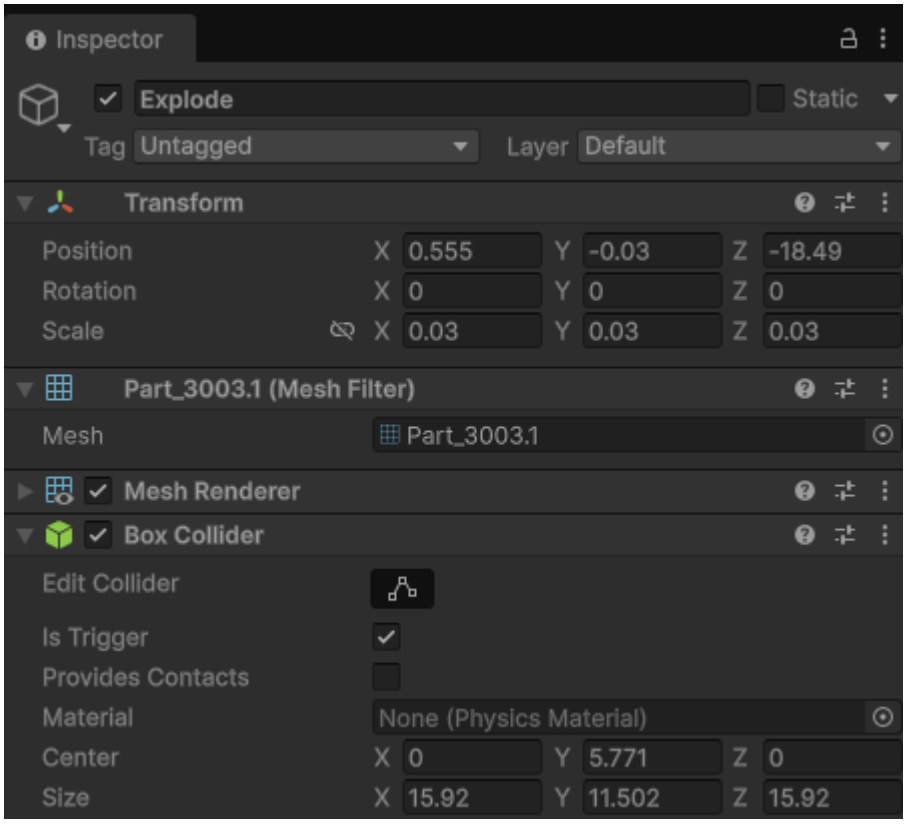



I added an animator to the object and set up the animations, setting the rest animation as the default and linking the explode to the restexplode so that it held its position once exploded. Whereas the implode was linked to rest so that it would stay imploded

W2117030
Matey Mateev




I then uploaded a 2x2 lego brick to act as a button trigger, I duplicated it twice and name each one Explode and Implode respectively, giving them box colliders and setting them as tiggers




I then had to type up a script to call the respective animations when the buttons were pressed;

W2117030
Matey Mateev

```
Assembly-CSharp                    BrickTriggerExplode                    brickArray
1   using UnityEngine;
2   using System.Collections;
3   using System.Collections.Generic;
4
5   // w2117030 Matey Mateev
    Unity Script (1 asset reference) | 0 references
6   public class BrickTriggerExplode : MonoBehaviour //class constructor
7   {
        Unity Serialized Field = 8BrickArrayExplodeFix | 1 reference
8       public GameObject brickArray;
        Unity Message | 0 references
9       void Start()
10      {
11
12      }
13
        Unity Message | 0 references
14      void Update()
15      {
16
17      }
        Unity Message | 0 references
18      public void OnMouseDown() //calls this on player click
19      {
20          brickArray.GetComponent<Animator>().Play("Explode"); //plays the "explode" animation when trigger is clicked
21      }
22  }
```

Explode

```
Assembly-CSharp                    BrickTriggerImplode                    OnMouseDo
1   using System.Collections;
2   using System.Collections.Generic;
3   using UnityEngine;
4
5   // w2117030 Matey Mateev

6   public class BrickTriggerImplode : MonoBehaviour
7   {

8       private GameObject brickArrayT;

9       void Start()
10      {
11          brickArrayT = GameObject.FindGameObjectWithTag("BrickArrayTag"); //finds specific game object with the BrickArrayTag Tag
12      }

13      void Update()
14      {
15      }
16
        Unity Message | 0 references
17      public void OnMouseDown()
18      {
19          brickArrayT.GetComponent<Animator>().Play("Implode"); //When the mouse is clicked on the trigger, plays Implode animation
20      }
21  }
```

Implode

I then added the respective scripts to the triggers and linked the array with the script so that it could access it. However my buttons didn't work!

I ran into this issue.

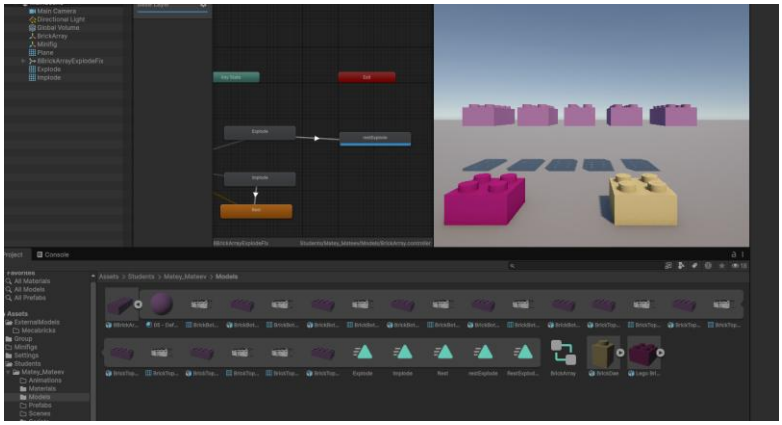



The error stated that there was no animator despite there being one in the brickArray object.

The solution was to drag the brick array from the hierarchy into the script trigger, as I had dragged the one from the assets menu which didn't have an animator.

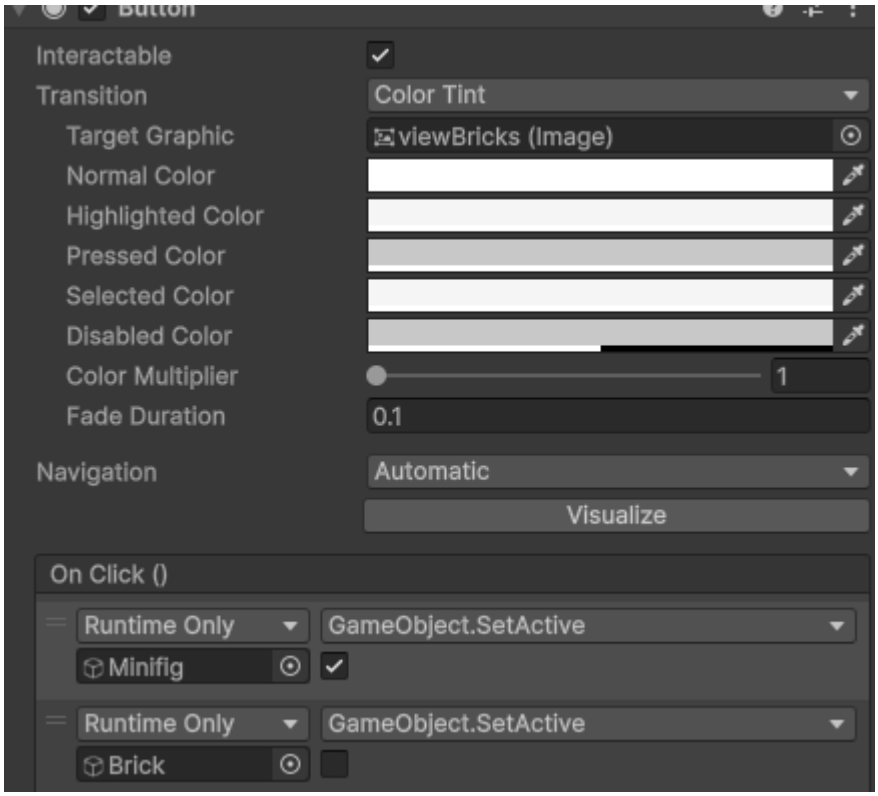
I also had an issue of the rest explode animation not staying once the explode played, this was because my length for the restexplode was 20-20, which meant the duration was 0 and unity got confused. I simply changed the restexplode animation to  20-21 to achieve this result

W2117030
Matey Mateev



I also had a small spelling issue with my implode tag. I had spelt it wrong in the script therefore it didn't find the brickArray. This was an easy fix though (screenshots were added last so this won't show)

I then needed a way to link these two cameras so that the user can switch between them I added 2 side buttons that toggle their respective cameras, so the "bricks" button would turn on the brickarray camera whilst turning off the minifig camera, vice versa. This allowed me to switch between the 2 cameras by pressing their buttoms



W2117030
Matey Mateev

After I set this up, my UI looked something like this




Which was fine, however when switching to the bricks camera, the walk button would stay, which made things pretty confusing.

After some research, I decided that I want to use a script to hide and show the walk button when the minifig button was pressed (as this is more scalable with a bigger project, good practice), I came up with this very simple script.

```
using UnityEngine;

//w2117030 Matey Mateev

public class ButtonVisibility : MonoBehaviour
{

    public GameObject walk;



    public void ShowWalkButton() // shows walk/stand button
    {
        walk.SetActive(true);
    }

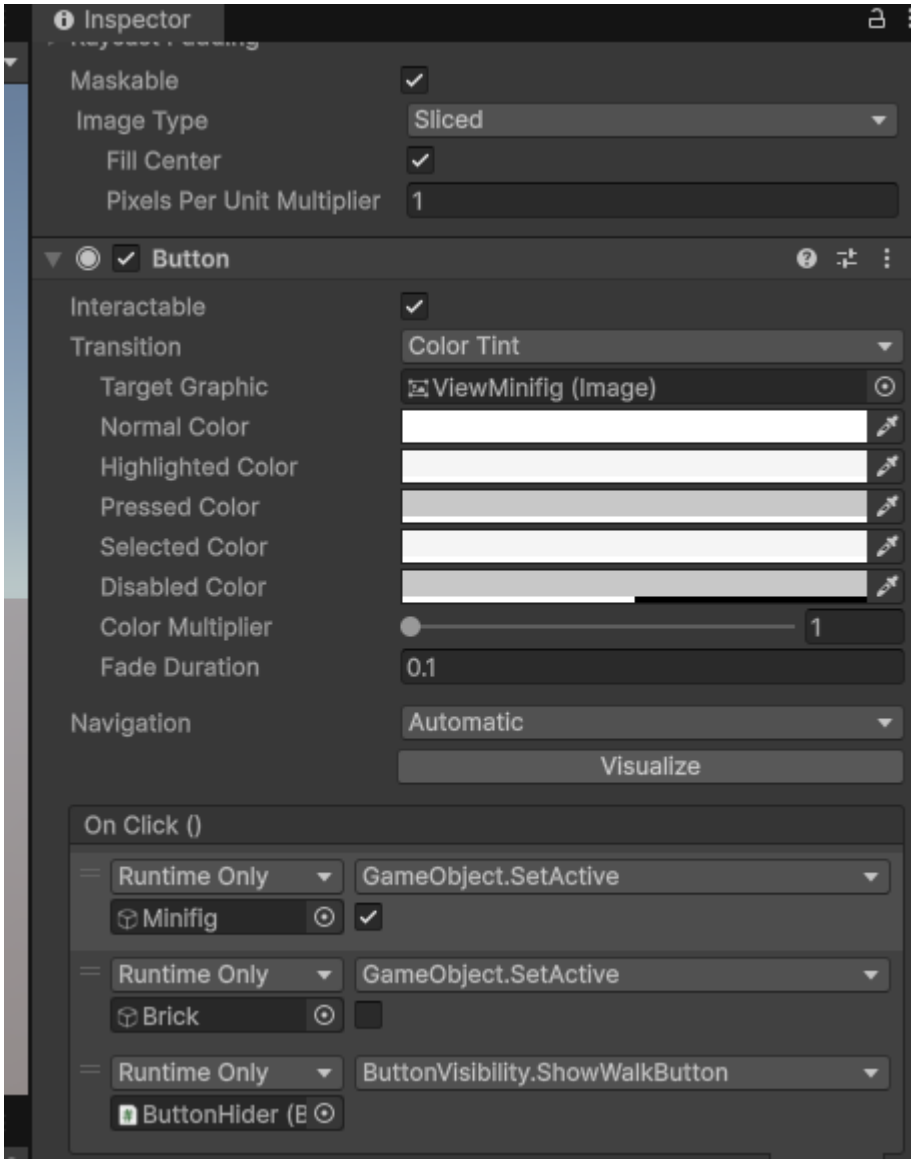


    public void HideWalkButton() //hides walk/stand button
    {
        walk.SetActive(false);
    }
}
```

Using the buttons' built in SetActive function to toggle it within the script.

I added another onclick() function to the bricks and minifig buttons to run this script.

W2117030
Matey Mateev




So now when the minifig button was pressed, the script would show the walk button, and when the bricks button was pressed, the script would hide the button.

**AI use log and Reflection**

| What I asked | AI Answer + How I used it | How it helped | What I Learned |
|---|---|---|---|
| **Asked AI to summarise the coursework brief into a checklist, including the mark scheme** | **Ai provided a structured checklist of what I needed to do, I used this to make sure I had everything completed** | **It helped me keep track of my work better as check-lists are easier for me to keep up with.** | **After checking against the actual mark scheme to make sure the AI didn't miss anything, I learned that AI is good for getting a new** |

| | | | |
|---|---|---|---|
| | | | **perspective on tasks** |
| **I asked AI to Generate a structure I should use for my files and folders in Unity** | **Ai gave me a layout of folders relevant to the coursework that I added to my project** | **Spent less time on organisation and allowed my work to stay tidy** | **Some folders weren't useful to me, so they were deleted** |
| **I asked AI what are some ways to toggle unity buttons when another one is pressed** | **AI gave me 2 options on how to do this, I asked for pros and cons of each and picked which one was more fitting** | **Took less time than figuring it out myself** | **AI overcomplicated the script in my opinion, so I re-did it my way which was way less code.** |

**Link to my GitHub folder in the repo**